# Homework Assignment 2

The second homework assignment is due on **October 11th at 11:59PM**. Please closely follow instructions listed in the syllabus.

Note: Use `print()` function to display output results after each part of homework problems. For example, `print(vec1)`.

**Problem 1 [20pt]**

While working on this problem, employ functions from the `apply()` family whenever you find the necessity for utilizing `loops`.

1. Develop a function named `fun1` that takes one parameter, `n`, denoting the number of trials in your simulation. The function should simulate the process of rolling two fair dice and report the outcomes of `n` trials. It should return vectors for the outcomes of both dice, labeled as `Die1` and `Die2`, respectively, along with a vector named `comparison`. The `comparison` vector is generated using the `mapply()` function and is a factor with three levels: "Die1 > Die2", "Die1 < Die2", and "Die1 = Die2". For example, if the result of Die1 is 5 and the result of Die2 is 4, the comparison should be "Die1 > Die2".

   Test the function with `n = 30`.

2. Construct a function named `fun2` with a single argument, `n`, representing the number of trials in your simulation. The function should simulate the process of rolling an unfair die with the following probability distribution: "1" - 0.1, "2" - 0.3, "3" - 0.2, "4" - 0.05, "5" - 0.3, "6" - 0.05. It should report the results of the `n` trials by returning a frequency table of the outcomes, indicating how many times each outcome (e.g., "1", "2", etc.) occurs.

   Test the function with `n = 100`.

3. Create a function named `fun3` that takes a list as input and performs the following operations:

   - Calculate the mean value for each element in the input list.
   - Reorder the elements in the list based on their mean values, arranging them in ascending order. The first element will have the smallest mean value, followed by the second element with the second smallest mean value, and so on.
   - Return the newly rearranged list.

Test the function with the provided list: list([20, 13, 34, 2], [2, 3], [45, 65, 78], [43, 98, 1, 22], [3, 5, 10], [13, 19, 14]).

4. Create a function called `fun4` with three parameters: `n`, `mu`, and `sigma` (where `n` is the number of randomly generated numbers, `mu` is the mean value of the distribution, and `sigma` is the standard deviation of that distribution). The function performs the following tasks:

   - Generate `n` random numbers from a normal distribution with mean = `mu` and standard deviation = `sigma`.
   - Populate an empty matrix based on the following conditions: If `n` is divisible by 2 but not by 3, fill the matrix by rows with 2 rows and `n`/2 columns. If `n` is divisible by 3 but not by 2, fill the matrix by columns with 3 columns and `n`/3 rows. If `n` is divisible by both 2 and 3, fill the matrix by rows with 6 rows and `n`/6 columns. Otherwise, populate the matrix by rows with only one column.
   - Return a vector containing the mean values of each column in the matrix if none of the randomly generated numbers are negative. If some of the generated numbers are negative, return a vector of the median values of each row in the matrix.

   Test the function with parameters (`n` = 50, `mu` = 0.5, `sigma` = 2).

**Problem 2 [25pt]**

Download the `carseats` dataset from Courseworks and import it into R (ensure to convert all character variables into factors). The dataset offers details on the sales of child car seats across 400 distinct stores, featuring the following variables:

- `Sales` - Unit sales (in thousands) at each location
- `Advertising` - Local advertising budget for company at each location (in thousands of dollars)
- `Population` - Population size in region (in thousands)
- `Price` - Price company charges for car seats at each site
- `ShelveLoc` - A factor with levels Bad, Good and Medium indicating the quality of the shelving location for the car seats at each site
- `Age` - Average age of the local population
- `Urban` - A factor with levels No and Yes to indicate whether the store is in an urban or rural location
- `US` - A factor with levels No and Yes to indicate whether the store is in the US or not

1. What is the average price the company charges for car seats? How does the average price for car seats in the US compare to that in non-US stores? Additionally, what is the average price for car seats in rural US stores compared to those in rural areas outside of the US?

2. Which store has the highest charge (provide the index number in the dataset)? Conversely, which store has the lowest charge?

3. What is the median number of unit sales for various quality levels of shelving locations? Now, respond to the same question by incorporating the `Urban` variable, resulting in six combinations.

4. Contrast the mean age of the local population between US and non-US stores by presenting boxplots. Customize the boxplots by incorporating labels, modifying colors, and adding notches for enhanced clarity.

5. Provide an overview of the quality of shelving locations using a barplot. Customize the presentation as outlined in the preceding question, excluding notches in this case.

6. Create side-by-side barplots illustrating the frequency of the `ShelveLoc` variable in both urban and rural areas. Tailor your plot according to the previous discussion.

7. Generate a plot of the `Sales` variable against the `Price` variable. Properly label the plot and axes. Adjust the point shape to triangular, with points colored in orange for urban stores and green for rural stores. Include a legend on the plot to indicate the respective areas. Modify the point size to 0.9 for clarity.

8. Integrate the histogram of the `Price` variable and the boxplot of the `Population` variable into a single image. Personalize both plots as detailed in part (4), excluding notches in the histogram.